

HC01 Ergebnisbericht

Ziel war es, einen prototypischen Smart Home Demonstrator zu entwickeln, der zentrale Anforderungen moderner Smart Home Infrastrukturen abbildet. In der Realität ergeben sich mehrere wiederkehrende Architekturprinzipien, an denen sich der entwickelte Demonstrator orientiert. Ein Smart Home besteht nicht nur aus einzelnen Geräten, sondern aus vielen Komponenten mit unterschiedlichen Rollen. Daraus ergeben sich verschiedene Herausforderungen: Die Geräte müssen miteinander kommunizieren können, obwohl sie unterschiedliche Aufgaben und Schnittstellen besitzen. Neue Geräte müssen in die Infrastruktur aufgenommen werden können, aber das sollte kontrolliert stattfinden. Gleichzeitig sollten sensible Daten geschützt werden, weil Smart Home Daten Rückschlüsse auf private Gewohnheiten, Anwesenheit oder Nutzungsmuster ermöglichen können. Eine moderne Architektur sollte außerdem erweiterbar bleiben, weil in einem Smart Home jederzeit neue Gerätetypen, Dienste oder Automationen ergänzt werden können.

Aus diesem Grund wurde für den Demonstrator ein klarer Schwerpunkt gewählt: sicheres Onboarding (Aufnahme) neuer Smart Home Geräte und verschlüsselte Kommunikation über einen nicht vertrauenswürdigen MQTT-Broker. Der Demonstrator soll nicht nur zeigen, dass Sensoren Daten senden und Aktoren gesteuert werden können. Er soll zusätzlich zeigen, dass ein Gerät vor der Aufnahme geprüft wird, nicht legitime Geräte abgelehnt werden und dass die Nachrichten, die über das Kommunikationsprotokoll verschickt werden Ende-zu-Ende verschlüsselt sind. Der Schwerpunkt passt zu den zentralen Anforderungen aus der Recherche. Moderne Smart Home Infrastrukturen müssen verteilt, erweiterbar, robust und heterogen sein. Gleichzeitig müssen sie Sicherheitsziele wie Authentifizierung, Integrität (Nachrichten können nicht unbemerkt verändert werden) und Vertraulichkeit (nur berechnigte Komponenten können die Nachrichten lesen) berücksichtigen. Genau diese Eigenschaften werden im Demonstrator vereinfacht, aber nachvollziehbar umgesetzt.

Ein bekanntes Kommunikationsmodell im IoT-Bereich ist das Publish Subscribe Prinzip. Dabei senden Geräte Nachrichten nicht direkt an einen konkreten Empfänger, sondern veröffentlichen sie auf einem „Topic“. Andere Komponenten können diese Topics abonnieren und erhalten dann die Nachrichten¹. MQTT setzt dieses Modell um und ist besonders für IoT Kontexte geeignet². Für Smart Home Systeme ist dieser Ansatz sinnvoll, weil Sensoren, Aktoren, usw. dadurch lose gekoppelt werden. Das ist ein Vorteil gegenüber direkter Kommunikation. Würde ein Sensor seine Daten direkt an bestimmte Zielkomponenten senden, müsste er jede dieser Zielkomponenten kennen. Wenn später ein neuer Automationsdienst ergänzt wird, müsste der Sensor angepasst werden, während er bei MQTT unverändert bleibt.

Der Broker (Mosquitto) wird im Demonstrator bewusst als nicht vertrauenswürdige Komponente betrachtet. Die Architektur setzt nicht voraus, dass der Broker sensible Nutzerdaten lesen darf oder vertrauenswürdige sein muss, er soll lediglich die Nachrichten weiterleiten. Die Inhalte der Nachricht werden zwischen den Geräten und dem Hub

¹ <https://www.hivemq.com/blog/mqtt-essentials-part-5-mqtt-topics-best-practices/>

² <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

verschlüsselt übertragen. Dadurch kann der Broker zwar sehen, dass Nachrichten übertragen werden, aber die Inhalte sind geheim.

Matter ist als IP basierter Standard für sichere IoT Systeme eine weitere interessante Entwicklung in der Smart Home Welt³. Ein Zentrales Konzept ist das Commissioning, was die kontrollierte Aufnahme eines neuen Geräts in ein Smart Home System meint. Das Gerät soll nachweisen, dass es ein echtes und zertifiziertes Gerät ist. Der Demonstrator bildet diese Idee nach: Neue Geräte werden nicht einfach akzeptiert, sondern müssen zuerst einen Onboarding Prozess durchlaufen. Dafür besitzen legitime Geräte eine eigene kryptografische Identität. Eine Certificate Authority stellt signierte Geräteinformationen, bzw. Zertifikate aus. Der Hub akzeptiert nur Geräte, deren Nachweis erfolgreich geprüft werden kann.

Bei Smart Home Architekturen unterscheidet sich der Verarbeitungsort zwischen Cloud und Edge. Cloud-Dienste können beispielsweise für Fernzugriff, langfristige Historie usw. genutzt werden, während Edge bedeutet, dass die Daten lokal im Heimnetz verarbeitet werden, beispielsweise durch einen Hub. Der Demonstrator bildet ein lokales Edge Smart Home ab.

Konkrete Anforderungen

- Das System soll simulierte Smart Home Geräte aufnehmen können (Erweiterbarkeit)
- Das System soll legitime von nicht legitimen Geräten unterscheiden können
- Jedes Gerät benötigt eine eindeutige Identität
- Neue Geräte müssen vor der Aufnahme verifiziert werden, nicht jedes Gerät darf automatisch Zugriff haben
- Ungültige Geräte werden abgelehnt
- Sensordaten sollen über MQTT übertragen werden können (Lose Kopplung)
- Aktoren sollen über definierte Topics gesteuert werden können
- Nutzerdaten sollen verschlüsselt übertragen werden
- Sicherheitsereignisse sollen protokolliert werden (Nachvollziehbarkeit)
- System soll realistische Smart Home Strukturen abbilden (Verteilt)
- Wenn ein Gerät ausfällt, soll der Rest des Systems weiterlaufen (Robustheit)

Architektur

Der Demonstrator ist als lokales Edge Smart Home aufgebaut. Alle Komponenten laufen lokal und kommunizieren über MQTT. Der Broker ist dabei die zentrale Nachrichtenverteilung, wird aber nicht als vertrauenswürdige Sicherheitsinstanz betrachtet. Die Sicherheit wird zwischen Gerät und Hub hergestellt.

Die Architektur besteht aus mehreren Komponenten. Die simulierten Geräte übernehmen die Rolle von Sensoren und Aktoren. Der MQTT-Broker verteilt Nachrichten über Topics. Der Hub verarbeitet Telemetrie, führt Automationslogik aus und steuert Aktoren. Die Certificate Authority stellt die Vertrauensbasis bereit. Der Commissioner übernimmt das Onboarding neuer Geräte. Das Gateway bindet ein Gerät mit abweichender Schnittstelle ein. Das Dashboard visualisiert den Systemzustand. Das Rogue Device simuliert einen Angriffsfall.

³ <https://csa-iot.org/all-solutions/matter/>

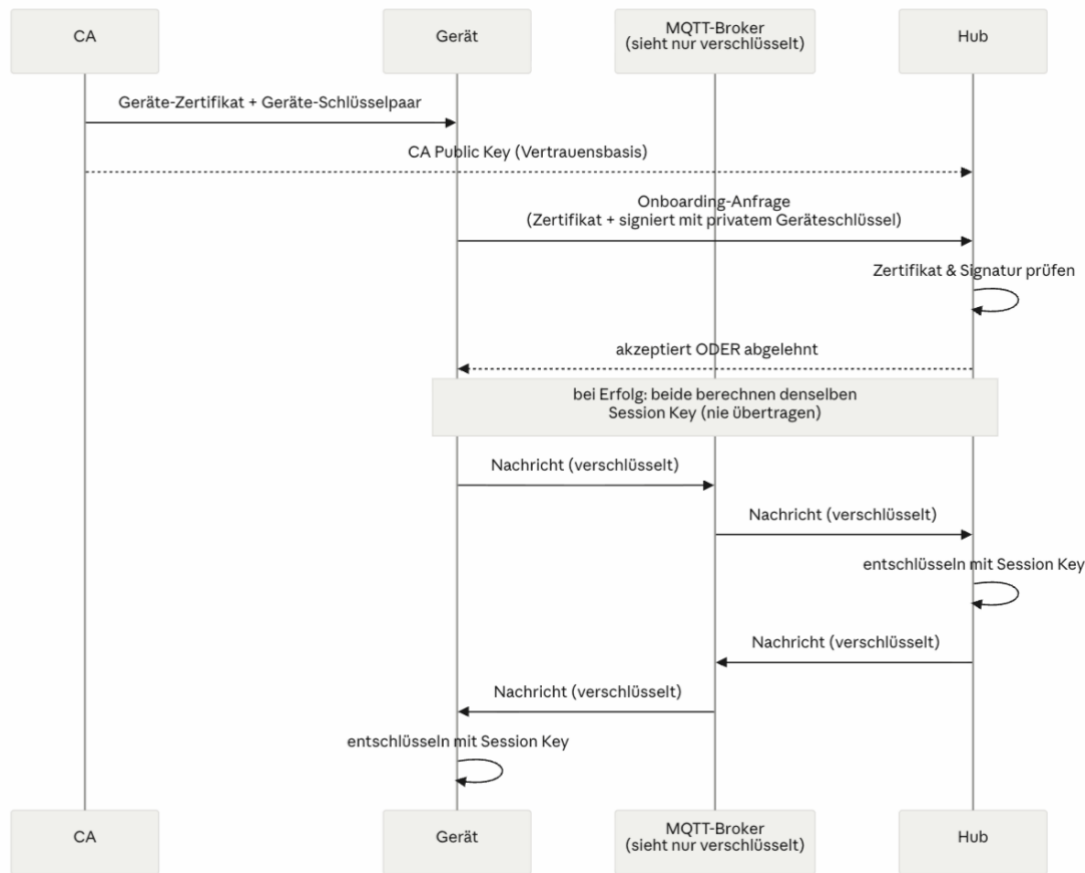
Komponente	Datei	Rolle
Certificate Authority (CA)	Ca/create_ca.py	Wird einmalig angelegt, Stellt dann Geräteidentitäten aus, signiert Geräte Zertifikate
MQTT-Broker	Broker/mosquitto.conf	Transport
Commissioner (Hub)	Hub/commissioner.py	Onboarding Prozess, prüft CA-Signatur, Proof-of-Possession, berechnet Session Key
Automation Service (Hub)	hub/automation_service.py	Reagiert auf Sensordaten, steuert Aktoren (verschlüsselt)
Heartbeat-Monitor (Hub)	Hub/heartbeat_monitor.py	Erkennt online/offline pro Gerät
Protokoll-Gateway (Hub)	Hub/gateway.py	Adapter für heterogenes http-Gerät
Sensoren	Devices/sensor.py	Erzeugt realistische Messreihen, sendet verschlüsselt
Aktoren	Devices/actuator.py	Empfängt verschlüsselte Befehle, setzt Zustand
http-Wetterstation	Devices/http_weather_station.py	Heterogenes Gerät, nur per http erreichbar
Heartbeat-sender	Devices/heartbeat_sender.py	Sendet regelmäßige Lebenszeichen
Rogue Device	Devices/rogue_device.py	Testet drei Angreifer Fälle
Broker Observer	Broker/broker_observer.py	Lauscher, für Dashboard Sicht
Dashboard	Dashboard/server.py	Web Anwendung
Krypto Klasse	Shared/crypto_utils.py	Für die Verschlüsselung (Ed25519, X25519, AES-GCM)
Topics	Shared/topics.py	Broker-Adresse, Topic-Namen
Nachrichten	Shared/messaging.py	Nachricht ver- und entschlüsseln
Onboarding	Shared/onboarding.py	Anfrage erstellen, Session finalisieren für Geräte
MQTT	Shared/mqtt_helpers.py	Client erzeugen, Sende-/Empfangsschleife

Die Infrastruktur ist konsequent verteilt aufgebaut, da jede Komponente als eigenständiger Prozess läuft. Die Certificate Authority, der MQTT-Broker, die Hub-Dienste sowie jedes einzelne Gerät sind separate Programme, die über ein Start-Skript als unabhängige Prozesse gestartet werden. Es gibt keinen gemeinsamen Speicher und keine direkten Funktionsaufrufe zwischen den Komponenten. Die gesamte Kommunikation läuft ausschließlich über Nachrichten, die der Broker vermittelt.

Crypto und Onboarding eines neuen Geräts

Beim Onboarding wird ein neues Gerät kontrolliert in die Infrastruktur aufgenommen, angelehnt an den Matter-Commissioning-Ansatz. Die vertrauenswürdige Instanz ist die Certificate Authority (CA), von der die Zertifikate für die Geräte erstellt und signiert werden. Das Signieren und die verschlüsselte Kommunikation basieren auf mehreren kryptografischen Verfahren. Die CA nutzt Ed25519-Signaturen. Dafür wird ein Schlüsselpaar erzeugt: ein Private Key, den nur die CA kennt, und ein Public Key, der öffentlich zugänglich ist. Wer den Private Key besitzt, kann signieren, mit dem Public Key lässt sich eine Signatur

verifizieren, ohne selbst signieren zu können. Das funktioniert, weil der Public Key aus dem Private Key abgeleitet ist: Aus dem Private Key lässt sich der Public Key leicht berechnen, in die umgekehrte Richtung ist das unmöglich (Einwegfunktion). Die erzeugten PEM-Dateien werden der Einfachheit halber unverschlüsselt im Dateisystem abgelegt, was in einem produktiven Setting natürlich nicht akzeptabel wäre. Die CA wird beim ersten Setup einmalig angelegt, wobei eine Kopie ihres Public Keys im Hub abgelegt wird. Dadurch kennt der Hub die CA als Trust Anchor (vertrauenswürdig). Jedes Gerät erhält ebenfalls ein eigenes Schlüsselpaar (Private und Public Key). Die CA erstellt das Gerätezertifikat (im JSON-Format) und setzt ihre Signatur darunter. Das signierte Device Certificate wird zusammen mit den Geräteschlüsseln in einem Ordner abgelegt. Für das eigentliche Onboarding schickt jedes Gerät, das aufgenommen werden möchte, eine Onboarding-Request an den Hub. Diese Anfrage signiert das Gerät mit seinem privaten Geräteschlüssel. So kann der Hub verifizieren, dass das Gerät tatsächlich den passenden privaten Schlüssel besitzt („Proof of Possession“). Gerät und Hub kommunizieren im laufenden Betrieb über das Netzwerk. Damit die spätere Kommunikation Ende-zu-Ende verschlüsselt laufen kann, brauchen beide einen gemeinsamen Session Key. Dieser kann nicht einfach über das Protokoll geschickt werden, da er dann nicht mehr geheim und die Verschlüsselung damit sinnlos wäre. Der Key muss also auf beiden Seiten unabhängig berechnet werden, ohne ihn zu übertragen. Dazu wird pro Onboarding-Prozess ein ephemeres (flüchtiges) Schlüsselpaar erzeugt, zunächst nur auf der Geräteseite. Verwendet wird X25519 für den Schlüsselaustausch und HKDF für die Ableitung des gemeinsamen Session Keys, da das rohe Shared Secret selbst nicht direkt als Schlüssel geeignet ist. In der Onboarding-Request schickt das Gerät u.a. das Gerätezertifikat und seinen ephemeren Public Key an den Hub und signiert die gesamte Anfrage mit seinem privaten Geräteschlüssel. Der Commissioner empfängt die Request und prüft, ob das Zertifikat echt ist und tatsächlich von der CA stammt. Das verifiziert er mithilfe des Public Key der CA. Anschließend prüft er, ob der Gerätetyp zugelassen ist und den Proof of Possession, also ob die Anfrage wirklich von dem Gerät stammt. Ist alles verifiziert, erzeugt der Hub sein eigenes ephemeres Schlüsselpaar und berechnet den Shared Key aus seinem privaten ephemeren Schlüssel und dem öffentlichen ephemeren Schlüssel des Geräts. Die Antwort des Hubs enthält dessen öffentlichen ephemeren Schlüssel und geht zurück an das Gerät. Das Gerät berechnet daraufhin den Shared Key aus seinem privaten ephemeren Schlüssel und dem öffentlichen Schlüssel des Hubs. Beide Seiten kommen so zum selben Session Key, ohne, dass er jemals übertragen wurde. Ein Fingerprint (Hash des Session Keys) wird auf beiden Seiten verglichen und bestätigt, dass die Ableitung übereinstimmt. Ab diesem Zeitpunkt nutzen Gerät und Hub den gemeinsamen Session Key mit AES-GCM, um Nachrichten zu verschlüsseln.



Rogue Device

Um ein böses Gerät zu simulieren, dass sich in den Hub einschleichen will, wurde das Rogue Device entwickelt. Es wurden drei verschiedene Szenarien betrachtet:

1. Das Rogue Device besitzt kein gültiges Gerätezertifikat und versucht sich mittels gefälschter CA selbst ein Zertifikat auszustellen.
2. Das Rogue Device nimmt ein echtes Zertifikat und passt den Inhalt an sein Gerät an.
3. Das Rogue Device kopiert ein echtes öffentliches Zertifikat

In Szenario 1 erstellt das Rogue Device einen eigenen ED25519-Key (quasi als Fälschung der CA) und signiert das Zertifikat selbst. Er stellt eine Onboarding-Request an den Hub, und erhält wie erwartet eine Ablehnung.

In Szenario 2 ist die Signatur echt, jedoch der Inhalt des Zertifikats verändert. Er stellt eine Onboarding-Request an den Hub, aber weil der Inhalt des Zertifikats gefälscht ist, wird er trotz CA-Signatur abgelehnt.

```
Commissioner: verarbeite offene Onboarding-Anfragen...

Anfrage: onboarding_request_rogue_device_01.json
Rejected: CA signature is invalid
-> Onboarding abgelehnt: Zertifikatsprüfung fehlgeschlagen!

Fertig: 0 von 1 Anfragen akzeptiert.
```

In Szenario 3 gibt sich das Rogue Device als anderes Gerät aus und kopiert das echte öffentliche Zertifikat eines Geräts aus dem hub. In diesem Fall scheitert das Onboarding am Proof of Possession, da das Rogue Device nicht den tatsächlichen private device key besitzt und die Anfrage somit mit einem falschen device key signiert wurde.

```
Commissioner: verarbeite offene Onboarding-Anfragen...

Anfrage: onboarding_request_rogue_stolen.json
Accepted: Certificate is valid(deviceId=temp_sensor_01, type=temperature_sensor)
-> Onboarding abgelehnt: Proof-of-Possession ungültig!

Fertig: 0 von 1 Anfragen akzeptiert.
```

Alle drei Szenarien zeigen, dass das System legitime von nicht legitimen Geräten zuverlässig unterscheidet: Unautorisierte Geräte werden bereits beim Onboarding abgewiesen. In Kombination mit der Ende-zu-Ende-Verschlüsselung des Betriebs adressiert der Demonstrator damit sowohl Authentizität (kein Fremdgerät kommt herein) als auch Vertraulichkeit (der Broker sieht nur verschlüsselte Nachrichten).

Sensoren, Aktoren, Automationslogik

Der Demonstrator umfasst mehrere simulierte Geräte in drei Rollen: Sensoren erfassen Zustände und veröffentlichen verschlüsselte Messwerte, Aktoren empfangen verschlüsselte Steuerbefehle und setzen ihren Zustand, und der Hub bildet die Automationslogik dazwischen. Konkret simuliert sind ein Temperatursensor, mehrere Bewegungsmelder (im Innenraum, am Eingang sowie im Außenbereich) und eine HTTP-Wetterstation, die die Luftfeuchte liefert. Als Aktoren existieren Licht, Heizung, Rollläden, Alarm und Lüftung. Die Sensoren erzeugen dabei keine reinen Zufallswerte, sondern verhalten sich möglichst realistisch. Die Automationslogik liegt zentral im Hub (Automation Service). Er abonniert über eine MQTT-Wildcard alle Telemetrie-Topics und verarbeitet damit auch zukünftige Sensortypen ohne Codeänderung. Aus den eingehenden Messwerten leitet er durch Regeln Steuerbefehle ab:

- Temperatur < 20 °C schaltet die Heizung ein, > 22 °C aus
- Bewegung im Innenraum schaltet bei Dunkelheit das Licht ein
- Ein Tür-Ereignis am Eingang setzt die Anwesenheit (Ankommen = anwesend, Verlassen = abwesend)
- Bewegung im Außenbereich löst den Alarm aus, sofern das System aktiv ist (niemand zuhause oder nachts)

- Luftfeuchte über 60 % schaltet die Lüftung ein, unter 50 % aus
- Der Rollladen wird rein zeitgesteuert geschlossen bzw. geöffnet (nachts zu)

Besonders anschaulich ist der Kreis zwischen Temperatursensor und Heizung. Wenn die Temperatur unter den Bereich fällt, schaltet der Hub die Heizung ein, woraufhin der vom Sensor gemeldete Wert wieder ansteigt, bis die Heizung erneut ausgeschaltet wird. Sensor, Hub und Aktor beeinflussen sich also gegenseitig, sodass sich der Wert realistisch um den Sollbereich einpendelt. Auch die Heterogenität ist konkret umgesetzt. Die Wetterstation spricht ausschließlich HTTP und beherrscht selbst keine Kryptografie. Ein Protokoll-Gateway onboardet stellvertretend für sie, ruft ihre Werte per HTTP ab, verschlüsselt sie und publiziert sie über MQTT. Aus Sicht des Hubs ist sie damit ein normales, sicheres Gerät. Hinsichtlich Skalierbarkeit und Erweiterbarkeit sind Sensoren und Aktoren jeweils generisch implementiert. Ein weiteres Gerät desselben Typs erfordert daher keine Codeänderung, sondern nur eine eigene Identität (device_id, Zertifikat, Onboarding).

Robustheit

Damit der Hub erkennt, welche Geräte überhaupt noch aktiv sind, sendet jedes Gerät in regelmäßigen Abständen ein Heartbeat, also ein kurzes Lebenszeichen. Dieses wird, wie jede Telemetrie mit dem Session Key des Geräts verschlüsselt und damit zugleich authentifiziert. Ein Angreifer kann kein gefälschtes „online“ für ein fremdes Gerät einschleusen, weil ihm der Schlüssel fehlt. Der Heartbeat-Monitor im Hub entschlüsselt jedes eingehende Lebenszeichen und merkt sich pro Gerät den Zeitpunkt des letzten gültigen Heartbeats. Bleibt ein Gerät länger als eine bestimmte Zeit (im Demonstrator 15 Sekunden) stumm, wird es als offline markiert.

Fazit

Zusammenfassend zeigt der Demonstrator, dass moderne Smart Home Infrastrukturen nicht nur einzelne Sensoren und Aktoren miteinander verbinden müssen, sondern einen kontrollierten und sicheren Rahmen für die Kommunikation benötigen. Durch das sichere Onboarding wird verhindert, dass sich unbekannte oder manipulierte Geräte unbemerkt in das System integrieren. Die Ende-zu-Ende verschlüsselte Kommunikation sorgt zusätzlich dafür, dass sensible Telemetriedaten auch über einen nicht vertrauenswürdigen Broker vertraulich bleiben. Insgesamt zeigt der Demonstrator, dass eine moderne Smart Home Architektur nicht allein durch funktionierende Gerätekommunikation entsteht, sondern durch das Zusammenspiel von Sicherheit, loser Kopplung, Erweiterbarkeit und Robustheit.